

Ahhh, got you now 😊

You mean **ALL Git commands that work locally — no GitHub/remote commands at all**. So exclude `git push`, `git pull`, `git remote`, GitHub authentication, etc.

Here is the **local Git command reference**, compactly written as `command` → `purpose`.



ALL LOCAL GIT COMMANDS

1. Git Basics / Configuration

<code>git --version</code>	→ Show Git version
<code>git help</code>	→ Show Git help
<code>git help <command></code>	→ Help for a command
<code>git <command> -h</code>	→ Short help
<code>git <command> --help</code>	→ Detailed help
<code>git config --list</code>	→ Show all configuration
<code>git config --global --list</code>	→ Show global configuration
<code>git config --local --list</code>	→ Show repository configuration
<code>git config --system --list</code>	→ Show system configuration
<code>git config user.name</code>	→ Show username
<code>git config user.email</code>	→ Show email
<code>git config --global user.name "Name"</code>	→ Set global username
<code>git config --global user.email "Mail"</code>	→ Set global email
<code>git config user.name "Name"</code>	→ Set repository username
<code>git config user.email "Mail"</code>	→ Set repository email
<code>git config --global --edit</code>	→ Edit global config
<code>git config --local --edit</code>	→ Edit local config
<code>git config --unset <key></code>	→ Remove configuration
<code>git config --global --unset <key></code>	→ Remove global configuration

2. Create / Initialize Repository

<code>git init</code>	→ Create Git repository
<code>git init <folder></code>	→ Initialize specific folder
<code>git init --bare</code>	→ Create bare repository
<code>git clone <path></code>	→ Clone local repository
<code>git clone <path> <folder></code>	→ Clone into specified folder

Note: `clone` can also work entirely locally when given a local filesystem path.

3. Repository Information

<code>git status</code>	→ Show repository status
<code>git status -s</code>	→ Short status
<code>git status --short</code>	→ Short status
<code>git status -b</code>	→ Show current branch
<code>git rev-parse --show-toplevel</code>	→ Show repository root
<code>git rev-parse --is-inside-work-tree</code>	→ Check if inside Git repository
<code>git rev-parse --git-dir</code>	→ Show <code>.git</code> directory
<code>git rev-parse HEAD</code>	→ Show current commit
<code>git rev-parse --abbrev-ref HEAD</code>	→ Show current branch

4. Add / Stage

<code>git add <file></code>	→ Stage file
<code>git add <file1> <file2></code>	→ Stage multiple files
<code>git add .</code>	→ Stage current directory changes
<code>git add -A</code>	→ Stage all changes
<code>git add --all</code>	→ Stage all changes
<code>git add -u</code>	→ Stage modified/deleted tracked files
<code>git add -i</code>	→ Interactive staging
<code>git add -p</code>	→ Stage changes piece by piece

git add -n → Dry run; show what would be added

5. Unstage

- git restore --staged <file> → Unstage file
 - git restore --staged . → Unstage everything
 - git reset <file> → Unstage file
 - git reset → Unstage everything
 - git reset HEAD <file> → Unstage file
-

6. Commit

- git commit → Create commit
 - git commit -m "message" → Commit with message
 - git commit -a -m "message" → Commit tracked changes
 - git commit -am "message" → Same as above
 - git commit --amend → Modify last commit
 - git commit --amend -m "message" → Change last commit message
 - git commit --amend --no-edit → Add changes without changing message
 - git commit --dry-run → Show what would be committed
 - git commit --allow-empty -m "msg" → Create empty commit
-

7. View Changes

- git diff → Unstaged changes
 - git diff --cached → Staged changes
 - git diff --staged → Staged changes
 - git diff HEAD → All changes since last commit
 - git diff <commit> → Compare working tree with commit
 - git diff <commit1> <commit2> → Compare two commits
 - git diff <branch1> <branch2> → Compare branches
 - git diff --stat → Show change statistics
 - git diff --name-only → Show changed filenames
 - git diff --name-status → Show filenames + status
 - git diff --word-diff → Show word-level changes
 - git diff --check → Check whitespace errors
-

8. Log / History

- git log → Full commit history
 - git log --oneline → One-line history
 - git log --stat → History + file statistics
 - git log -p → History + actual changes
 - git log --graph → Graph history
 - git log --decorate → Show branch/tag references
 - git log --all → Show all references
 - git log --oneline --graph --all → Compact visual history
 - git log -n 5 → Show last 5 commits
 - git log -5 → Show last 5 commits
 - git log --since="1 week ago" → Commits from time period
 - git log --until="2026-01-01" → Commits before date
 - git log --author="Name" → Commits by author
 - git log --grep="text" → Search commit messages
 - git log -- <file> → History of specific file
 - git show → Show latest commit
 - git show <commit> → Show specific commit
 - git show <commit>:<file> → Show file from commit
 - git show --stat <commit> → Commit statistics
-

9. Branches

git branch	→ List branches
git branch -a	→ List all branches
git branch -v	→ Branch + latest commit
git branch -vv	→ Branch + tracking info
git branch <name>	→ Create branch
git branch -d <name>	→ Delete merged branch
git branch -D <name>	→ Force delete branch
git branch -m <new-name>	→ Rename current branch
git branch -m <old> <new>	→ Rename branch
git switch <branch>	→ Switch branch
git switch -c <branch>	→ Create + switch branch
git switch -C <branch>	→ Force create + switch
git checkout <branch>	→ Switch branch (older syntax)
git checkout -b <branch>	→ Create + switch branch

10. Merge

git merge <branch>	→ Merge branch
git merge --no-ff <branch>	→ Force merge commit
git merge --ff-only <branch>	→ Allow only fast-forward
git merge --squash <branch>	→ Combine changes without commit
git merge --abort	→ Cancel merge
git merge --continue	→ Continue merge
git merge --no-commit <branch>	→ Merge without automatic commit
git merge --edit <branch>	→ Edit merge message

11. Merge Conflict

git status	→ Find conflicted files
git diff	→ View conflicts
git add <file>	→ Mark conflict resolved
git add .	→ Mark all resolved
git merge --continue	→ Continue merge
git merge --abort	→ Cancel merge

12. Reset

git reset	→ Unstage everything
git reset <file>	→ Unstage file
git reset --soft HEAD~1	→ Undo commit, keep staged changes
git reset --mixed HEAD~1	→ Undo commit, keep changes unstaged
git reset --hard HEAD~1	→ Undo commit + delete changes ⚠
git reset --soft <commit-id>	→ Move HEAD, keep changes staged
git reset --mixed <commit-id>	→ Move HEAD, keep changes unstaged
git reset --hard <commit-id>	→ Move HEAD + discard changes ⚠

Remember:

--soft	→ Commit removed, changes STAGED
--mixed	→ Commit removed, changes UNSTAGED
--hard	→ Commit removed, changes DELETED ⚠

13. Revert

git revert <commit-Id>	→ Reverse a commit
git revert HEAD	→ Reverse latest commit
git revert --no-commit <commit>	→ Reverse without committing
git revert --continue	→ Continue revert
git revert --abort	→ Cancel revert

Unlike **reset**, **revert** creates a **new commit**.

14. Restore

- git restore <file> → Discard working changes
 - git restore . → Discard all working changes

 - git restore --staged <file> → Unstage file
 - git restore --staged . → Unstage everything

 - git restore --source=<commit> <file> → Restore file from commit
 - git restore --source=HEAD <file> → Restore from latest commit

 - ⚠ `git restore <file>` can permanently discard uncommitted changes.
-

15. Stash

- git stash → Save current changes
 - git stash push → Save current changes
 - git stash push -m "message" → Save with name
 - git stash -u → Include untracked files
 - git stash -a → Include ignored files too

 - git stash list → List stashes
 - git stash show → Show latest stash
 - git stash show -p → Show stash changes

 - git stash apply → Apply latest stash
 - git stash apply stash@{0} → Apply specific stash

 - git stash pop → Apply + remove latest stash
 - git stash pop stash@{0} → Apply + remove specific stash

 - git stash drop → Delete latest stash
 - git stash drop stash@{0} → Delete specific stash
 - git stash clear → Delete all stashes

 - git stash branch <branch> → Create branch from stash
-

16. Tags

- git tag → List tags
 - git tag <name> → Create tag
 - git tag -a <name> -m "message" → Create annotated tag
 - git tag <name> <commit> → Tag specific commit

 - git show <tag> → Show tag information
 - git tag -d <tag> → Delete tag
-

17. Cherry-pick

- git cherry-pick <commit> → Apply commit to current branch
 - git cherry-pick <commit1> <commit2> → Apply multiple commits
 - git cherry-pick A..B → Apply commit range

 - git cherry-pick --no-commit <commit> → Apply without committing
 - git cherry-pick --continue → Continue cherry-pick
 - git cherry-pick --abort → Cancel cherry-pick
 - git cherry-pick --skip → Skip current commit
-

18. Rebase

- git rebase <branch> → Rebase current branch
- git rebase <commit> → Rebase onto commit

- git rebase -i HEAD~3 → Interactive rebase
- git rebase --continue → Continue rebase

git rebase --abort

→ Cancel rebase

git rebase --skip

→ Skip commit

git rebase --onto <new> <old>

→ Rebase onto another base

Interactive commands:

pick

→ Keep commit

reword

→ Change commit message

edit

→ Edit commit

squash

→ Combine with previous commit

fixup

→ Combine, discard message

drop

→ Delete commit

19. Reflog 🔥

git reflog

→ Show HEAD movements

git reflog --all

→ Show all reflogs

git reflog show

→ Show reflog

git reflog expire

→ Expire old reflog entries

Extremely useful for recovering "lost" commits.

20. File Tracking

git ls-files

→ List tracked files

git ls-files -s

→ Show staging information

git ls-files --others

→ Show untracked files

git ls-files --deleted

→ Show deleted files

git ls-files --modified

→ Show modified files

git ls-files --ignored

→ Show ignored files

21. Remove Files

git rm <file>

→ Delete + stage deletion

git rm -r <folder>

→ Delete folder + stage deletion

git rm --cached <file>

→ Stop tracking file

git rm -f <file>

→ Force remove file

22. Clean Untracked Files

git clean -n

→ Preview deletion

git clean --dry-run

→ Preview deletion

git clean -f

→ Delete untracked files

git clean -fd

→ Delete untracked files/directories

git clean -fx

→ Include ignored files

git clean -fdx

→ Delete everything untracked/ignored

⚠️ **Very dangerous:** `git clean -fdx`

23. Ignore Files

Git itself doesn't have a `git ignore` command. You create:

.gitignore

→ Define ignored files

Useful Git commands:

git check-ignore <file>

→ Check whether file is ignored

git check-ignore -v <file>

→ Show why file is ignored

24. Blame

- git blame <file> → Show who changed each line
 - git blame -L 10,20 <file> → Blame lines 10–20
 - git blame -w <file> → Ignore whitespace changes
 - git blame <commit> -- <file> → Blame at specific commit
-

25. Search

- git grep "text" → Search tracked files
 - git grep -n "text" → Search + line numbers
 - git grep -i "text" → Case-insensitive search
 - git grep "text" <commit> → Search inside commit
-

26. Bisect

Used to find **which commit introduced a bug**.

- git bisect start → Start bisect
 - git bisect bad → Mark current commit bad
 - git bisect good <commit> → Mark commit good
 - git bisect good → Mark current commit good
 - git bisect bad → Mark current commit bad
 - git bisect skip → Skip commit
 - git bisect reset → Finish bisect
 - git bisect log → Show bisect history
-

27. Worktree

Work on multiple branches simultaneously.

- git worktree list → List worktrees
 - git worktree add <path> <branch> → Create worktree
 - git worktree add -b <branch> <path> → Create branch + worktree
 - git worktree remove <path> → Remove worktree
 - git worktree move <old> <new> → Move worktree
 - git worktree prune → Clean invalid worktrees
 - git worktree lock → Lock worktree
 - git worktree unlock → Unlock worktree
-

28. Submodules

For repositories inside repositories:

- git submodule add <path> → Add submodule
 - git submodule init → Initialize submodules
 - git submodule update → Update submodules
 - git submodule update --init → Initialize + update
 - git submodule update --recursive → Update nested submodules
 - git submodule status → Show submodule status
 - git submodule deinit <path> → Deinitialize submodule
-

29. Subtree

- git subtree add → Add subtree
 - git subtree pull → Pull subtree changes
 - git subtree push → Push subtree changes
 - git subtree split → Split subtree history
 - git subtree merge → Merge subtree
-

30. Notes

git notes add	→ Add note to commit
git notes show	→ Show commit note
git notes edit	→ Edit note
git notes list	→ List notes
git notes remove	→ Remove note

31. Maintenance / Repository Internals

git gc	→ Clean/optimize repository
git gc --aggressive	→ More aggressive cleanup
git fsck	→ Check repository integrity
git count-objects	→ Count Git objects
git prune	→ Remove unreachable objects
git reflog expire	→ Expire reflog entries
git repack	→ Repack repository
git verify-pack	→ Verify packed objects

32. Object Commands

Git internally stores objects as:

blob	→ File content
tree	→ Directory structure
commit	→ Snapshot + metadata
tag	→ Annotated tag object

Commands:

git cat-file -t <object>	→ Show object type
git cat-file -p <object>	→ Show object contents
git cat-file -s <object>	→ Show object size
git hash-object <file>	→ Create hash of file
git hash-object -w <file>	→ Store object in Git database

33. Show References

git show-ref	→ Show references
git for-each-ref	→ List references
git rev-parse HEAD	→ Get current commit
git rev-parse <branch>	→ Get branch commit
git rev-parse <tag>	→ Get tag commit

34. Describe

git describe	→ Describe current commit
git describe --tags	→ Describe using tags
git describe --always	→ Always produce description

35. Archive

Create an archive from Git:

git archive HEAD	→ Create archive of current commit
git archive --format=zip HEAD	→ Create ZIP archive
git archive --format=tar HEAD	→ Create TAR archive

36. Apply Patches

git apply <patch>	→ Apply patch
git apply --check <patch>	→ Check patch
git apply --reverse <patch>	→ Reverse patch

37. Format Patches

- git format-patch <commit> → Create patch files
 - git format-patch -1 → Patch for latest commit
 - git format-patch -3 → Patches for last 3 commits
-

38. Import Patches

- git am <patch> → Apply email patch
 - git am --continue → Continue
 - git am --skip → Skip
 - git am --abort → Cancel
-

39. Shortcuts / Aliases

- git config --global alias.st status → Create "git st"
 - git config --global alias.co checkout → Create "git co"
 - git config --global alias.br branch → Create "git br"
 - git config --global alias.cm commit → Create "git cm"

 - git config --get-regexp alias → Show aliases
 - git config --unset alias.st → Remove alias
-

40. Environment / Debugging

- git var -l → Show Git variables
 - git var GIT_AUTHOR_IDENT → Show author identity
 - git var GIT_COMMITTER_IDENT → Show committer identity

 - git version --build-options → Git build information
 - git diagnose → Generate diagnostic information
-

41. Advanced Repository Commands

- git update-index → Manipulate index
- git read-tree → Read tree into index
- git write-tree → Write index as tree
- git commit-tree → Create commit object
- git mktree → Build tree object
- git unpack-objects → Unpack objects
- git index-pack → Build pack index
- git pack-objects → Create pack
- git unpack-file → Extract blob

These are mostly **Git internals**, not everyday commands.

42. Git Plumbing Commands

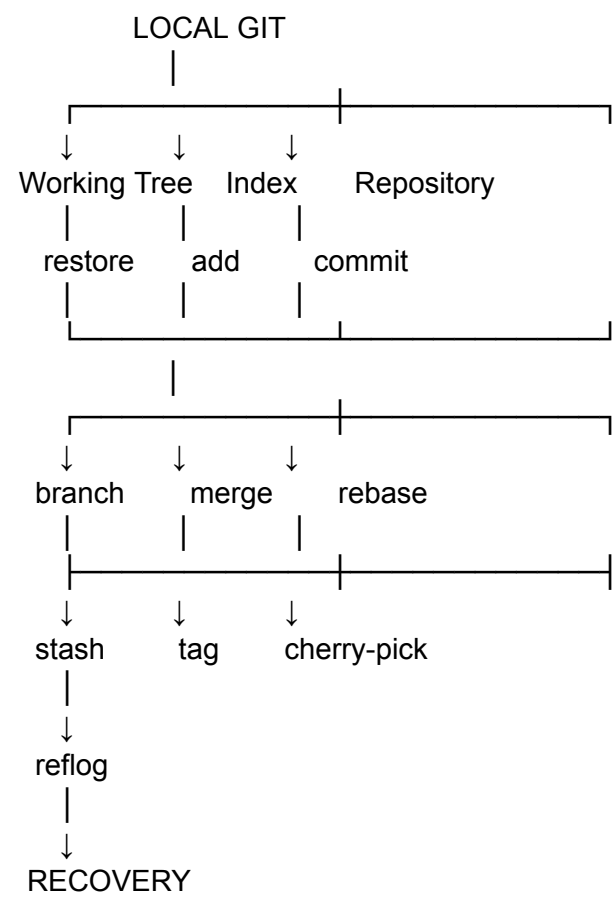
For completeness — these are the lower-level Git commands:

- git apply
- git archive
- git bisect
- git cat-file
- git check-attr
- git check-ignore
- git check-mailmap
- git check-ref-format
- git checkout-index
- git commit-graph
- git count-objects
- git diff-files
- git diff-index
- git diff-tree
- git for-each-ref

git hash-object
git index-pack
git ls-files
git ls-tree
git merge-base
git mktag
git mktree
git multi-pack-index
git name-rev
git pack-objects
git prune
git read-tree
git reflog
git remote-ext
git remote-fd
git remote-file
git replace
git rev-list
git rev-parse
git show-index
git show-ref
git symbolic-ref
git unpack-file
git unpack-objects
git update-index
git update-ref
git verify-pack
git verify-commit
git verify-tag
git write-tree

The Big Picture

If we strip Git down to its local core, the important concepts are:



★ If by "ALL" you literally mean all commands

Git has **hundreds of commands/subcommands**, including plumbing/internal commands and commands that are platform-specific or rarely used. The list above covers the **full practical local command set plus the major plumbing commands**, while intentionally leaving out **remote/GitHub operations** such as:

- ✗ git push
- ✗ git pull
- ✗ git fetch
- ✗ git remote
- ✗ git ls-remote
- ✗ git clone from a remote URL

The **most important local commands** to master first are:

- git init → Create repository
- git status → Check state

- git add → Stage changes
- git commit → Save snapshot
- git diff → Compare changes
- git log → View history
- git branch → Manage branches
- git switch → Switch branches
- git merge → Merge branches
- git rebase → Reorganize history
- git stash → Temporarily save work
- git restore → Undo working changes
- git reset → Move/reset history
- git revert → Reverse a commit
- git tag → Mark versions
- git cherry-pick → Copy a commit
- git reflog → Recover lost commits
- git clean → Remove untracked files
- git bisect → Find buggy commit
- git blame → Find line history

That is the **local-Git-only** version — no GitHub workflow mixed in.